

第19章 多态性及应用

19.1 多态性

封装和信息隐藏、继承和多态是面向对象的三大基本特性。多态性(Polymorphism)意为多种形态,指在程序中,对于相同的请求消息,可以存在多个不同的处理方式。在 C++中,相同的消息通常体现为同名成员函数,而不同的处理方式则对应于不同的函数实现。当出现多态时,尽管多个函数可能拥有相同的名称,但它们的行为和执行结果却各不相同。多态性虽然允许定义多个函数体,但在实际执行时只会选择其中一个。根据确定执行哪个函数体的方式不同,多态性可分为静态多态性和动态多态性。

19.1.1 静态多态

静态多态是根据目标对象的静态类型和参数表中参数的静态类型确定目标代码体。在 C++中,函数重载和模板就体现了静态多态性。例:

```
1 //静态多态-重载-例
2 #include <iostream>
3 class A {
4 public:
5     void f(int a) { std::cout << "Int"<< std::endl;}
6     void f(char c) { std::cout << "Char"<< std::endl;}
7 };
8 int main( ){
9     A obj;
10    obj.f( 2 ); //编译时确定调用 A::f( int )的函数体,执行时输出 Int
11    obj.f('c'); //编译时确定调用 A::f( char )的函数体,执行时输出 Char
12 }
```

这里,向 A 类对象 obj 发送消息 f,根据实参的静态类型,在编译时就确定了各自的调用函数,这个过程在编译期完成,所以是静态多态。再如:

```
1 //静态多态-模板-例
2 #include <iostream>
3 #include <typeinfo>
4 using namespace std;
5 template <typename T>
6 class My {
7 public:
```

```

8      void f(const T& t)
9          { cout << typeid(t).name( ) << endl; }
10 };
11 int main( ) {
12     My<int> obj1;
13     My<float> obj2;
14     obj1.f(2);    //输出 T 的类型名, 如 i
15     obj2.f(2);    //输出 T 的类型名, 如 f
16 }

```

这里, `My` 是一个类模板, `My<int>` 和 `My<float>` 是通过实例化该类模板生成的两个不同类。尽管在源代码中函数 `f` 只在一个地方定义, 但 `My<int>` 和 `My<float>` 分别拥有自己的 `f` 函数及相应的函数体, 且这两个函数的参数列表和参数类型也有所不同。因此, 在编译时, `obj1.f(2)` 会被编译为 `obj1.My<int>::f(2)`, 而 `obj2.f(2)` 则会被编译为 `obj2.My<float>::f(2)`。运行时, 这两个调用会执行不同的函数体, 这体现了静态多态性。

19.1.2 动态多态

相比于静态多态在编译期确定目标代码, 动态多态则是在运行时才确定具体的目标代码。动态多态可分为以下两种情况:

- 根据目标对象的动态类型和参数表中参数的静态类型来确定目标代码。例如, 虚机制。
- 根据目标对象的动态类型和参数表中参数的动态类型来确定目标代码。目前 C++ 尚不支持这种形式。

实际上, 第二种动态多态才是真正意义上的动态多态, 但由于其在语言设计和编译器实现方面存在较大难度, 当前大多数面向对象语言尚未提供支持。因此, 我们通常所说的动态多态一般指的是第一种情况, 而虚函数是实现这种动态多态的一种常见方式。例如:

```

1  class Parent {
2  public:
3      virtual void func( ) { std::cout<< 0 << std::endl; }
4      virtual ~Parent( ) { }
5  };
6  class Child1 : public Parent {
7  public:
8      virtual void func( ) override { std::cout<< 1 << std::endl; }
9  };
10 class Child2 : public Parent {
11 public:
12     virtual void func( ) override { std::cout<< 2 << std::endl; }

```

```

13 };
14 int main( ) {
15     Parent * p1 = new Child1;
16     Parent * p2 = new Child2;
17     p1->func( ); //执行 Child1::func( ), 输出 1
18     p2->func( ); //执行 Child2::func( ), 输出 2
19     delete p2;
20     delete p1;
21 }

```

对于 `p1->func( )`, `p1` 的动态类型为 `Child1*`, 因此在运行时会调用 `Child1::func( )`。同样地, 对于 `p2->func( )`, `p2` 的动态类型为 `Child2*`, 所以在运行时会调用 `Child2::func( )`。由此可见, 在虚函数机制下, 实际执行的代码取决于接收对象的动态类型, 这正是动态多态性的体现。需要注意的是, 在虚函数机制中, 参数匹配是基于实参的静态类型进行的。例如:

```

1  #include <iostream>
2  class Child;
3  class Parent {
4  public:
5      virtual void func( Parent & obj) { std::cout<< 1 << std::endl; }
6      virtual void func( Child & obj) { std::cout<< 2 << std::endl; }
7      virtual ~Parent( ) {}
8  };
9  class Child : public Parent {
10 public:
11     virtual void func( Parent & obj) override { std::cout<< 3 << std::endl; }
12     virtual void func( Child & obj) override { std::cout<< 4 << std::endl; }
13 };
14 int main( ) {
15     Child child;
16     Parent & parent = child;
17     parent.func( parent ); //输出 3
18     parent.func( child ); //输出 4
19     child.func( parent ); //输出 3
20     child.func( child ); //输出 4
21 }

```

在这里, `parent` 的静态类型是 `Parent`, 而动态类型是 `Child`。编译 `parent.func(parent)` 时, 根据 `parent` 的静态类型在 `Parent` 类中匹配到虚函数 `Parent::func(Parent&)`。编译器会在编译时确定该函数在 `Parent` 类的虚拟表中的索引号, 并在运行时根据该索引号查找 `Child` 类的虚拟表中对应的函数地址, 即 `Child::func(Parent&)` 的地址, 因此执行时会输出 3。其他语句的输出原理与此类似。

19.2 重载的使用

使用静态多态通常体现为函数重载和模板的应用。从严格意义上讲，模板属于泛型编程的范畴，因此这里我们主要讨论函数重载的使用。

在继承关系中，派生类中的函数可能会隐藏基类中的同名函数。为了提高基类的复用性，在基类中应谨慎使用重载。由于在定义一个类时通常无法预知该类是否会作为基类被继承，因此在类设计中一般不推荐使用重载。只有当类中存在多个非虚函数且它们的语义相似，并且可以确定此类不会被继承时，才应考虑使用重载。因此，具有重载函数的类更适合作为终端类(**final**)。例如：

```
1  class A final {
2      public:
3          void add( char );
4          void add( int );
5          void add( int,int,int );
6          void add( double val, int precision );
7          void add( const Complex & c);
8  };
```

类 A 由于被 **final** 修饰符限定，成为不可继承的 **final** 类，因此无法从类 A 派生新类，从而避免了成员被隐藏的风险。对于这些重载的 **add** 函数，由于参数列表较为简单，各函数的行为含义直观且语义差异易于理解，通常不会给类的使用者带来困扰。然而，如果参数列表变得更加复杂，或者 **add** 表示的是专业领域内的特定行为，那么正确理解和使用这些重载函数可能会变得困难。

在这种情况下，最好的解决办法是消除函数重载。为此，可以采用以下两种方法：

1. 使用更具描述性的函数名称来替代重载函数。通过为每个不同的函数赋予明确的名称，不仅消除了重载，还使函数的语义更加清晰易懂。
2. 对重载函数的参数列表进行抽象化处理，将多个重载函数合并为一个统一的函数。例如，可以通过重新设计上边的类 A 来实现这一点，例如：

```
1  class A {
2      public:
3          //将参数表抽象为 Number 类，不同参数表对应于 Number 的不同子类
4          void add( const Number & num);
5          //或者将数值参数抽象为 Number 类，参数个数和精度抽象到 Option 类中
6          //void add( const Number & num, const Option & opt);
7          //或者数值参数抽象为 Number 类，参数个数和精度抽象仍作为形参
8          //void add( const Number & num, int numCount = 1, int precision = 0);
9  };
```

在类中，还有一类特殊的重载函数——重载的构造函数。由于构造函数的名称必须与类名相同，当存在多个构造函数时，只能以重载的形式定义。如果构造函数的数量固定且希望更直观地区分它们的语义，也可以通过类方法为构造函数提供不同的名称。例如：

```
1 class Balloon {
2     public:
3         Balloon( );
4         Balloon( int size, int color);
5     };
```

可以重新设计为

```
1 class Balloon {
2     public:
3         static Balloon createNormalBalloon( );
4         static Balloon createSpecialBalloon( int size, int color);
5     private:
6         Balloon( );
7         Balloon( int size, int color);
8     };
```

19.3 虚机制的使用

在软件设计与开发过程中，除了关注软件功能外，还应高度重视软件的复用性。软件复用是指在构建新软件系统时，对现有软件产品(如设计架构、源代码、文档和用例等)进行有效再利用的技术。如果不考虑软件复用，讨论面向对象程序设计将失去意义。尽管软件复用涉及多个层面，但在本课程中，我们将主要探讨程序代码中类的复用，而动态多态性正是提升类复用性的关键机制之一。

相较于静态多态，动态多态的应用更为广泛，通常狭义上的多态即指动态多态。在 C++ 语言中，动态多态是通过虚机制实现的。动态多态的价值在于：它能够在保持客户端访问接口不变的情况下变更类的实现。这里的“客户端”可以是一个函数、一个类、一个模块或一个完整的程序。借助动态多态，可以在确保既有类复用性的基础上灵活应对需求变化。例如：

```
1 class Parent {
2     public:
3         virtual ~Parent( ) { }
4         virtual void func( ) { /* 实现 1 */ }
5     };
6 class Child1 : public Parent {
7     public:
8         virtual ~Child1( ) { }
9         virtual void func( ) override { /* 实现 2 */ }
```

```

10 };
11 class Child2 : public Parent {
12 public:
13     virtual ~Child2( ) {}
14     virtual void func( ) override { /* 实现 3 */ }
15 };
16 class Client {
17 public:
18     Client( Parent & obj) : mpObject(&obj) { }
19     void useObject(Parent & obj) {
20         obj.func( );
21     }
22     void usePointer( ) {
23         mpObject->func( );
24     }
25 private:
26     Parent * mpObject;
27 };
28 int main( ) {
29     Child1 child1;
30     Child2 child2;
31     Client client(child1);
32     client.useObject(child2);
33     client.usePointer( );
34 }

```

这里，Child1 和 Child2 是 Parent 类的两个子类，各自对 void func() 接口有不同的实现；而 Client 类是访问 Parent 继承体系的客户端。当需要更改 func() 的实现时，可以通过派生新的子类 Childn 来实现。例如：

```

1 class Childn : public Parent {
2 public:
3     virtual ~Childn( ) { }
4     virtual void func( ) override { /* 新的实现 n */ }
5 };

```

同时修改 int main() 函数，

```

1 int main( ) {
2     Child1 child1;
3     Childn childn;          //修改 Child2 child2; 为 Childn childn;
4     Client client(child1);
5     client.useObject(childn); //修改 child2 为 childn
6     client.usePointer( );
7 }

```

这时可以看到：为了适应变更 `func()` 实现的需求，引入了新的类 `Childn`，同时对 `main` 函数进行了修改，但未对 `Parent`、`Child1`、`Child2` 和 `Client` 类的任何内容进行改动。这确保了在此次修改 `main` 函数之前使用这些类的既有代码仍然可以正常工作。通过仅增加新类而不修改原有类，只要 `Client` 中访问的 `Parent` 接口保持不变，`Client`、`Parent`、`Child1` 和 `Child2` 类就不需要修改，从而保证了这些类的良好复用性。这正是动态多态的价值所在。

可能会有人提出疑问：虽然复用了 `Client` 类，但仍需修改 `main()` 函数的实现，这种复用是否还有实际意义？

首先，随着需求变化，程序的行为和输出理应随之调整。C++ 作为静态编译型语言，任何功能变更最终都需落实到源代码层面，这可能涉及类的扩展、接口的调整，或 `main()` 函数逻辑的变更，甚至多处协同修改。关键不在于“是否修改”，而在于“如何修改更合理”。遵循**最小修改原则**（Principle of Least Modification），即在满足新需求的前提下，尽可能减少对已有稳定模块的影响，才是工程实践中的核心考量。从复用性的角度看，仅调整 `main()` 的调用逻辑，而完全保留 `Client` 类的原有结构与职责，恰恰体现了高内聚、低耦合的设计思想，是复用价值的真实体现。

其次，若新需求导致 `main()` 变得臃肿或频繁变动，正说明其承担了过多职责。此时，不应视之为复用失效，而应将其视为重构契机：可通过进一步抽象，将业务流程封装为独立的控制器（如 `Application` 或 `Runner` 类）、引入配置驱动或应用设计模式，甚至采用命令行参数解析等机制，使 `main()` 退化为简洁的入口代码。例如，可将原示例中硬编码的客户端调用逻辑提取到独立的 `AppMgr` 类，从而提升可维护性与可扩展性：

```

1  //Parent,Child1,Child2,Client 等类略
2  class AppMgr {
3  public:
4      AppMgr(Parent & o1,Parent & o2) : mClient(o1),mpObject(&o2) { }
5      void do1( ) { mClient.useObject( mObject ); }
6      void do2( ) { mClient.usePointer( ); }
7  private:
8      Client mClient;
9      Parent * mpObject;
10 };
11 int main( ) {
12     Child1 child1;
13     Childn childn;
14     AppMgr mgr(child1,childn);
15     mgr.do1( );
16     mgr.do2( );

```

```
17 }
```

当需要新增子类 ChildX 时, Parent、Child1、Child2 和 Client 类以及新抽象的 AppMgr 类均无需修改, 仅需调整 main 函数即可。

从上述分析可以看出, 当一个类的外部访问接口保持不变时, 可以通过将该接口定义为虚函数, 并在子类中提供不同的具体实现, 从而适应接口实现的变化需求, 同时保持类的复用性。通常, 当函数实现呈现出以下形式时, 就可以考虑应用虚函数机制进行重新设计:

```
1  class A {
2  public:
3      void func( Parameters & params) {
4          if( condition( ) == 1 ) {
5              //实现 1
6          } else if ( condition( ) == 2 ) {
7              //实现 2
8          } else if ( condition( ) == 3 ) {
9              //实现 3
10         } else if ( ... ){
11             //实现...
12         } else if ( condition( ) == N ) {
13             //实现 n
14         }
15     }
16     Data * getData( ) const { return mpData; }
17 private:
18     int condition( ) const { /* 略 */ }
19 private:
20     Data * mpData;
21 }
```

在上述代码中, 存在使用分支语句实现不同功能的需求。如果这些分支条件是互斥的, 尤其是在未来可能需要增加新的分支条件及其对应实现时, 可以考虑采用虚机制来适应这种需求。具体来说: 先将不同的实现进行抽象并封装成多个类, 然后从这些类中抽象出一个公共父类。这个公共父类可以是具体类、抽象类或接口类。接着, 在父类中定义公共行为接口, 并在各子类中提供具体的不同实现。

例如, 可以将实现 1、实现 2、...、实现 n 分别抽象为 FuncImp01 类、FuncImp02 类、...、FuncImp0N 类, 再定义这些类的公共父类 IFunc。在 IFunc 中定义公共接口, 如 virtual void execute(...), 并在其子类 FuncImp01、FuncImp02、...、FuncImp0N 中分别实现该接口。这样, 不仅

使代码结构更加清晰，也便于后续扩展和维护。

```

1  class A;
2  class Parameters;
3  class IFunc;
4  class IFunc {
5  public:
6      virtual void execute( A & a, Parameters & params ) = 0;
7  };
8  class A {
9  public:
10     void func( IFunc & imp , Parameters & params ) {
11         imp.execute(*this,params);
12     }
13     Data * getData( ) const { return mpData;}
14 private:
15     Data * mpData;
16 };
17 class FuncImp01:public IFunc {
18 public:
19     virtual void execute(A & a, Parameters & params) override {
20         //实现 1, 实现中可使用 params 和 a.getData( )
21     }
22 }
23 class FuncImp02:public IFunc {
24 public:
25     virtual void execute(A & a, Parameters & params) override {
26         //实现 2, 实现中可使用 params 和 a.getData( )
27     }
28 }
29
30 class FuncImp0N:public IFunc {
31 public:
32     virtual void execute(A & a, Parameters & params) override {
33         //实现 n, 实现中可使用 params 和 a.getData( )
34     }
35 }

```

可以看出，当需要添加新的实现时，只需定义一个实现了 IFuncImp 接口的新子类即可。这样，在新增实现时，只需增加新类，无需修改 A 类、Parameters 类、IFuncImp 接口以及已有的 FuncImp0X 类，从而有效提高了既有类的复用性。

因此，虚机制的本质在于通过子类型化来灵活应对需求的变化。在程序设计中，子类型化可应

用于多个方面，如函数实现、数据成员和参数列表等。以下代码就是在函数参数中应用子类型化的一个示例：

```

1  class Parent {
2  public:
3      virtual ~Parent( ) { }
4      virtual void func( ) { /* 实现 0 */ }
5  };
6  class Child1 :public Parent {
7  public:
8      virtual void func( ) override { cout << "1" << endl; /* 实现 1 */ }
9  };
10 class Child2 :public Parent {
11 public:
12     virtual void func( ) override { cout << "2" << endl; /* 实现 1 */ }
13 };
14 class ChildN :public Parent {
15 public:
16     virtual void func( ) override { cout << "n" << endl; /* 实现 N */ }
17 };
18 class Client {
19 public:
20     Client( Parent & parent1 )
21         : mpParent1( &parent1 ), mpParent2( &parent1 ) {
22     }
23     void setParent2( Parent & parent2 ) {
24         mpParent2 = &parent2;
25     }
26     void useAsParameters( Parent & parent3 ) {
27         //实例化的子类型对象用于参数列表
28         parent3.func( );
29     }
30     void useAsMember1( ) {
31         mpParent1->func( );
32     }
33     void useAsMember2( ) {
34         mpParent2->func( );
35     }
36 private:
37     Parent * mpParent1; //实例化的子类型对象用于数据成员
38     Parent * mpParent2; //实例化的子类型对象用于数据成员
39 };
40 int main( ) {
41     Child1 c1;

```

```

42     Child2 c2;
43     ChildN cn;
44     Client user( cn );
45     user.useAsParameters( c1 );    //对应实现 1
46     user.useAsMember1( );         //对应实现 n
47     user.setParent2( c2 );
48     user.useAsMember2( );         //对应实现 2
49 }

```

在这里, `Parent` 是父类, 而 `Child1`、`Child2` 等是 `Parent` 类的子类或后裔类。对于函数 `void Client::useAsParameters(Parent&)`, 传入的实际参数可以是任何 `Parent` 类的子类或后裔类类型, 即使实参是新定义的子类型, `Client` 类也无需进行修改。

19.4 应用例

19.4.1 软件开发类

现需要定义各种软件(`Software`)类, 它们的主要行为是开发(`develop`)。如果不考虑迭代的过程, 软件开发的基本流程都是由设计、编码、测试、维护等几个基本过程组成, 不同种类软件的开发过程只是这些基本过程的不同。因此, 可这样定义:

```

1  class Software {
2  public:
3      Software( const std::string & softwareName ) { /* 略 */ }
4      virtual ~Software( ) { }
5      void develop( ) {
6          design( );
7          coding( );
8          test( );
9          maintain( );
10     }
11 protected:
12     virtual void design( ) = 0;
13     virtual void coding( ) = 0;
14     virtual void test( ) = 0;
15     virtual void maintain( ) = 0;
16 };
17 class AppSoftware : public Software {
18 public:
19     AppSoftware( const std::string & softwareName ) :Software( softwareName ) { }
20 protected:
21     virtual void design( ) override { /* 略 */ }

```

```

22     virtual void coding( ) override    { /* 略 */ }
23     virtual void test( ) override     { /* 略 */ }
24     virtual void maintain( ) override { /* 略 */ }
25 };
26 class WebSoftware : public Software {
27 public:
28     WebSoftware( const std::string & softwareName ) :Software( softwareName ) { }
29 protected:
30     virtual void design( ) override    { /* 略 */ }
31     virtual void coding( ) override     { /* 略 */ }
32     virtual void test( ) override       { /* 略 */ }
33     virtual void maintain( ) override   { /* 略 */ }
34 };
35 int main( ) {
36     Software * psw1 = new AppSoftware( "微信" );
37     psw1->develop( );
38     delete psw1;
39     Software * psw2 = new WebSoftware( "教育部官网" );
40     psw2->develop( );
41     delete psw2;
42 }

```

本例中，`Software::develop()`定义了软件开发的过程框架，具体各过程的实现细节由子类自行决定。这样既保证了 `Software` 类复用性，又允许增加新类型的软件。新增类型时，既可以从 `Software` 派生，也可以从 `AppSoftware` 类或 `WebSoftware` 类派生。

19.4.2 虚拟拷贝构造

下面例子是关于虚拟拷贝构造函数的，也称虚拟构造函数。假设有一个盛有多个水果的水果篮，水果的种类可以有多种，如苹果、芒果等，现希望随意拿出一个水果，并复制出一个一样的水果。初步的类设计，如下：

```

1  class Fruit;
2  class Basket {
3  public:
4      Fruit * copy(int index);
5  private:
6      Fruit * mFruits[5];
7  };
8  class Fruit {
9  public:
10     virtual ~Fruit( ) {}

```

```

11 };
12 class Apple:public Fruit {
13 };
14 class Mango:public Fruit {
15 };

```

这里的关键是复制的行为，因为被复制的水果样本是任意的，也就是其类型可以是水果类型的任意子类或后裔类，而复制结果必须是同类型的。最直接的实现是：

```

1  Fruit * Basket::copy(int index) {
2      Fruit * fruit = mFruits[index];
3      if( (Apple * pApple = dynamic_cast<Apple*>(fruit)) ) {
4          return new Apple(*pApple);
5      }else if( Mango * pMango = dynamic_cast< Mango *>(fruit)) {
6          return new Mango(*pMango);
7      }else {
8          return nullptr;
9      }
10 }

```

但这样的实现，当增加水果种类时，必须在 `copy` 函数中增加新的条件分支，也就是需要修改 `copy` 函数的既有代码，这会破坏 `Basket` 类的复用性。如果 `Fruit` 类的拷贝构造函数可以是虚的，那么要解决此问题就很简单，即：

```

1  Fruit * Basket::copy(int index) {
2      Fruit * fruit = mFruits[index];
3      return new fruit->拷贝构造函数(*fruit);
4  }

```

可是在 C++ 语言中，拷贝构造函数不允许是虚的，而且各类的拷贝构造函数名称也互不相同，也就不能这样实现。为此，可以考虑在 `Fruit` 类中增加一个自定义的虚函数，其含义是完成类似拷贝构造函数的同样功能，具体的拷贝功能可在 `Fruit` 的后裔类中实现。重新设计和定义，如下：

```

1  class Fruit {
2  public:
3      Fruit( ) = default; //显式定义了拷贝构造函数，因此需显式定义构造函数
4      virtual ~Fruit( ) {}
5      virtual Fruit * clone( ) = 0;
6  protected: //子类的拷贝构造函数需要访问父类的拷贝构造
7      Fruit( const Fruit & ) = default; //防止外部访问拷贝构造函数
8  };
9  class Apple : public Fruit {
10 public:

```

```

11     Apple( ) = default; //显式定义了拷贝构造函数，因此需显式定义构造函数
12     virtual Apple * clone( ) override { return new Apple(*this); }
13 protected:
14     Apple( const Apple & ) = default; //防止外部访问拷贝构造函数
15 };
16 class Mango : public Fruit {
17 public:
18     Mango( ) = default; //显式定义了拷贝构造函数，因此需显式定义构造函数
19     virtual Mango * clone( ) override { return new Mango(*this); }
20 protected:
21     Mango( const Mango & ) = default; //防止外部访问拷贝构造函数
22 };
23 class Basket {
24 public:
25     Fruit * copy(int index) {
26         Fruit * fruit = mFruits[index];
27         return fruit->clone( );
28     }
29 private:
30     Fruit * mFruits[5];
31 };

```

可以看出，Fruit 类及后裔类中的 clone 函数都是虚函数，完成的功能就是各类的拷贝构造函数功能。这里 clone 函数就是虚拟拷贝构造函数，也称虚拟构造函数。使用虚拟构造函数，一方面，在复制样本时，不用关心样本的具体类型，使用方便，另一方面，易于扩展样本类型，并保持既有类的复用性。虚拟构造函数虽然使用方便，但它要求各后裔类必须有可访问的拷贝构造函数，这一点对于复杂类型就不太友好。

在实现虚拟拷贝构造函数时，应特别注意：1)各类的拷贝构造函数应显式定义并将其访问控制设为 protected。这样可以强制用户使用 clone 函数进行拷贝构造，而不是使用普通的拷贝构造函数，同时又允许本类以及子类访问拷贝构造函数。2)需显式定义构造函数。由于显式定义了拷贝构造函数，而拷贝构造函数也是一种构造函数，因此满足 C++11 标准的编译器将不再提供缺省的构造函数。

19.4.3 怪物战斗

一个游戏中有多种怪物(Monster)，怪物之间可能要发生战斗(fight)，每场战斗都是一个怪物与另一怪物之间的一对一战斗。每个怪物都有自己的生命值(hitpoint)、攻击力值(damage)和防御力值(defence)，每种怪物都有各自特有的攻击(attack)方式，产生相应的攻击效果；战斗时，两个怪物依

次攻击对方，即怪物 a 首先攻击怪物 b，然后轮到怪物 b 攻击怪物 a，之后，怪物 a 再次攻击怪物 b，...，直到一方生命值为 0。作为怪物的特例，猫和狗的攻击效果如下表所示。

表 19-1 进攻方导致对方生命值减少量

进攻方	数值
猫进攻导致对方的生命值减少量	(猫的攻击力值 * 2 - 对方的防御力值) 若上式小于 1，则取 1
狗进攻导致对方的生命值减少量	(狗的攻击力值 - 对方的防御力值 + 5) * 2 若上式小于 2，则取 2

根据给出的描述，请给出 Monster 类和 Dog、Cat 类的定义和实现(不用考虑怪物生命值为 0 后的处理)。

分析：Monster 的主要数据成员有生命值、攻击力和防御力，主要行为就是战斗(fight)，战斗的实现过程是一个无限循环，直到决出胜负为止。战斗中的攻击方式是各种 Monster 的差异所在，不同的 Monster 子类(狗、猫等)实现各自不同的进攻效果。各类之间的关系如图 19-1。

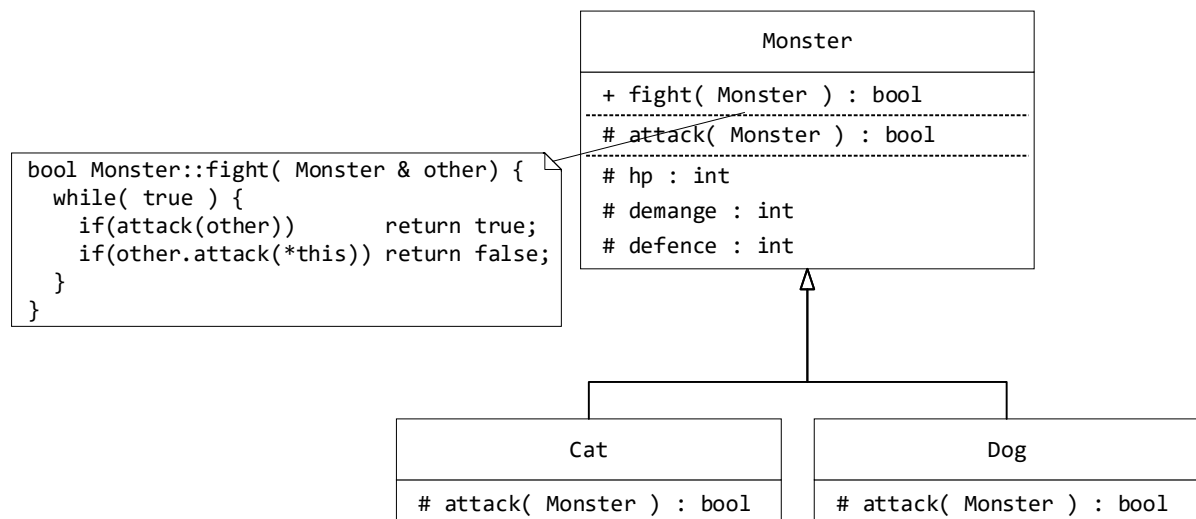


图 19-1 Monster 及子类的 UML 图

具体的定义和实现可如下：

```

1  class Monster {
2      static const bool WIN = true;
3      static const bool LOSE = false;
4  public:
5      Monster( int hp, int damage, int defence )
6          :mHp( hp ), mDamage( damage ), mDefence( defence ) {
7      }
8      virtual ~Monster( ) { }
9      bool fight( Monster & other ) {
10         while ( true ) {
  
```

```

11         if ( attack( other ) ) return WIN;
12         if ( other.attack( *this ) ) return LOSE;
13     }
14 }
15 protected:
16     virtual bool attack( Monster & other ) = 0;
17 public:
18     int getHp( ) const        { return mHp; }
19     void setHp( int newHp )   { mHp = ( newHp > 0 ? newHp : 0 ); }
20     int getDamage( ) const    { return mDamage; }
21     int getDefence( ) const   { return mDefence; }
22 private:
23     int mHp;                 //生命值
24     int mDamage;             //攻击力
25     int mDefence;            //防御力
26 };
27 class Cat : public Monster {
28 public:
29     Cat( int hp, int damage, int defence ) :Monster( hp, damage, defence ) { }
30 protected:
31     virtual bool attack( Monster & other ) override {
32         int harm = getDamage( ) * 2 - other.getDefence( );
33         if ( harm < 1 ) harm = 1;
34         other.setHp( other.getHp( ) - harm );
35         return ( other.getHp( ) <= 0 );    //other 生命减至 0, 返回 true, 表示胜利
36     }
37 };
38 class Dog : public Monster {
39 public:
40     Dog( int hp, int damage, int defence ) :Monster( hp, damage, defence ) { }
41 protected:
42     virtual bool attack( Monster & other ) override {
43         int harm = ( getDamage( ) - other.getDefence( ) + 5 ) * 2;
44         if ( harm < 2 ) harm = 2;
45         other.setHp( other.getHp( ) - harm );
46         return ( other.getHp( ) <= 0 );    //other 生命减至 0, 返回 true, 表示胜利
47     }
48 };

```

19.4.4 公司类

现有一个公司类(Company), 数据为全部职员的集合, 行为包括排序(sort)。Company 可能如下:

```

1 class Company {

```

```

2 public:
3     void sort( );
4 private:
5     Stuff * mpStuff;
6 };

```

如果未来需要变更排序的算法，为提高 Company 的复用性，

方法一：可以重新设计 Company 类，并从 Company 派生子类

```

1 class Company {
2 public:
3     virtual ~Company( ) { }
4     virtual void sort( );
5 protected:
6     Stuff * mpStuff;
7 };
8 class CompanySortAlgorithm01 :public Company {
9 public:
10     virtual void sort( ) { /* 排序算法 01 */}
11 };
12 class CompanySortAlgorithm02 :public Company {
13 public:
14     virtual void sort( ) { /* 排序算法 02 */}
15 };
16 ...

```

方法二：重新设计 Company，将 sort 功能的实现委托给 SortAlgorithm 类，从 SortAlgorithm 类派生不同的算法实现。

```

1 class Company {
2 public:
3     void sort( SortAlgorithm & alg ) {
4         alg.sort( *mpStuff );
5     }
6 private:
7     Stuff * mpStuff;
8 };
9 class SortAlgorithm {
10 public:
11     virtual ~SortAlgorithm( ) { }
12     virtual void sort( Stuff & stuff ) = 0;
13 };
14 class SortAlgorithm01: public SortAlgorithm {
15 public:

```

```

16     virtual void sort( Stuff & stuff) { /* 排序算法 01 */ }
17 };
18 class SortAlgorithm02: public SortAlgorithm {
19 public:
20     virtual void sort( Stuff & stuff) { /* 排序算法 02 */ }
21 };

```

无论哪种方法，都能够适应排序算法的改变，但考虑到 `Company` 类的各行为，除了排序还会有其它的功能行为，因此，方法二更具一般性。

进一步地，如果员工集合(`Stuff`)的组织结构也可能有多种，如可能是数组、单向链表、双向链表、树、甚至是稀疏矩阵，还能适应吗？这里先给出结论：如果员工集合的组织结构和排序算法的种类都不固定，虚机制也不能适应。

在本章的第一节中提到，动态多态包括两种：1)根据目标对象的动态类型和参数表中参数的静态类型确定目标代码体，2)根据目标对象的动态类型和参数表中参数的动态类型确定目标代码体。如果 C++ 支持第二种动态多态，那么成员函数 `void sort( Stuff & stuff)` 就能自动根据 `SortAlgorithm` 的不同类型和 `Stuff` 的不同类型，执行适当的实现代码，但目前各编程语言通常都不支持第二种动态多态。而只使用第一种动态多态，也就是虚机制，`sort(Stuff &)` 的实现可能是这样：

```

1 void SortAlgorithm0X::sort(Sutff & suff) {
2     if( sutff 是数组 ) {
3
4     }else if (stuff 是单向链表) {
5
6     } else if ( ... ) {
7     }
8 }

```

那么当增加新的组织结构时，必然要修改多个 `SortAlgorithm` 类的子类的实现代码，这不符合复用的思想。

可见，使用虚机制只能适用单个维度的实现变化。

19.4.5 导弹模拟

现欲开发一款导弹飞行仿真系统，该系统主要模拟不同型号导弹的发射方式与飞行特征。目前需要模拟的导弹型号及其特征如表 19-2 所示(模拟实现各行为时，输出字符串即可)。

表 19-2 导弹型号、发射方式及飞行特征

导弹型号(Missile)	发射方式(LaunchMode)	飞行特征(FigBehavior)
中华(ChinaA)	海基型(SeaLaunch)	亚音速飞行(SubSonicFly)
红鸟(RedBirdB)	空基型(AirLaunch)	亚音速飞行(SubSonicFly)
东风(SpringC)	陆基型(LandLaunch)	超音速飞行(SuperSonicFly)
远征(ExpeditionD)	海基型(SeaLaunch)	超音速飞行(SuperSonicFly)

未来还将模拟更多型号的导弹，其发射方式和飞行特征的组合可能与现有列表不同。请设计 Missile 类及相关类的定义，确保在添加新型号导弹时，无需修改现有的类和代码。

分析：针对上述问题的设计时，可定义 LaunchMode 和 FlyBehavior 作为接口或抽象类，分别表示发射方式和飞行特征；然后再定义 Missile 类，作为导弹的基类。例如：

```

1  #include <iostream>
2  //发射方式接口
3  class LaunchMode {
4  public:
5      virtual ~LaunchMode( ) = default; // 虚析构函数
6      virtual void launch( ) const = 0; // 纯虚函数，定义发射方式
7  };
8  //具体发射方式
9  class SeaLaunch : public LaunchMode {
10 public:
11     void launch( ) const override {
12         std::cout << "海基发射";
13     }
14 };
15 class AirLaunch : public LaunchMode {
16 public:
17     void launch( ) const override {
18         std::cout << "空基发射";
19     }
20 };
21 class LandLaunch : public LaunchMode {
22 public:
23     void launch( ) const override {
24         std::cout << "陆基发射";
25     }
26 };
27 //飞行方式接口
28 class FlyBehavior {
29 public:
30     virtual void fly( ) const = 0;    // 纯虚函数，定义飞行特征
31     virtual ~FlyBehavior( ) = default; // 虚析构函数
32 };

```

```

33 //具体飞行方式
34 class SubSonicFly : public FlyBehavior {
35 public:
36     void fly( ) const override {
37         std::cout << "亚音速飞行";
38     }
39 };
40 class SuperSonicFly : public FlyBehavior {
41 public:
42     void fly( ) const override {
43         std::cout << "超音速飞行";
44     }
45 };
46 //导弹类
47 class Missile {
48 public:
49     Missile( const std::string & name ) : mName( name ) {}
50     void simulate( LaunchMode & launchMode, FlyBehavior & flyBehavior ) {
51         std::cout << mName << "导弹: ";
52         launchMode.launch( );
53         std::cout << " - ";
54         flyBehavior.fly( );
55         std::cout << std::endl;
56     }
57 private:
58     std::string mName;
59 };
60 //模拟主函数
61 int main( ) {
62     // 创建导弹对象
63     Missile * missiles[ ] { new Missile( "中华" ), new Missile( "红鸟" ),
64                             new Missile( "东风" ), new Missile( "远征" ) };
65     // 创建发射方式
66     LaunchMode * modes[ 3 ] { new SeaLaunch( ), new AirLaunch( ), new LandLaunch( ) };
67     // 创建飞行特征
68     FlyBehavior * behaviors[ ] { new SubSonicFly( ), new SuperSonicFly( ) };
69     // 模拟导弹行为
70     // 输出: 中华导弹: 海基发射 - 亚音速飞行
71     missiles[ 0 ]->simulate( *modes[ 0 ], *behaviors[ 0 ] );
72     // 输出: 红鸟导弹 : 空基发射 - 亚音速飞行
73     missiles[ 1 ]->simulate( *modes[ 1 ], *behaviors[ 0 ] );
74     // 输出: 东风导弹 : 陆基发射 - 超音速飞行
75     missiles[ 2 ]->simulate( *modes[ 2 ], *behaviors[ 1 ] );
76     // 输出: 远征导弹 : 海基发射 - 超音速飞行

```

```

77     missiles[ 3 ]->simulate( *modes[ 0 ], *behaviors[ 1 ] );
78     //释放
79     for (auto p : missiles) delete p;
80     for (auto p : modes) delete p;
81     for (auto p : behaviors) delete p;
82 }

```

在当前设计中，导弹类的模拟行为依赖于发射方式和行为特征。我们也可以选择使用普通关联、聚合或组合的方式来关联发射方式和行为特征。例如，采用聚合关系的导弹类可以这样定义：

```

1  class Missile {
2  public:
3      Missile(const std::string& name, LaunchMode* launchMode, FlyBehavior* flyBehavior)
4          : mName(name), mpLaunchMode(launchMode), mpFlyBehavior(flyBehavior) {}
5      void launch( ) const {
6          std::cout << mName << "导弹:";
7          mpLaunchMode->launch( );
8      }
9      void fly( ) const {
10         std::cout << mName << "导弹:";
11         mpFlyBehavior->fly( );
12     }
13 private:
14     std::string  mName;
15     LaunchMode  * mpLaunchMode;
16     FlyBehavior * mpFlyBehavior;
17 };

```

19.4.6 字符输出

客户请小王编写一个从键盘读入字符并输出到打印机的程序。小王使用结构化的设计方法，编写了如下代码。

```

1  void copy( ) {
2      int c;
3      while ((c = readKeyboard( )) != EOF) // EOF 为宏定义的终结字符
4          writePrinter(c);
5  }

```

该程序包含 3 个子程序，copy 子程序从 readKeyboard 子程序中获取字符，并把字符传递给 writePrinter 子程序。几个月后，客户说有时希望 copy 程序能从纸带读入机中读入字符。并且，在未来可能还会从其它种类的输入设备中读入字符，也可能输出到其它种类的输出设备。小王希望他

写出的代码既能满足上述要求,又不用每次都改写 `copy` 的实现。请你帮小王重新设计 `copy` 函数,添加必要的类,使得新设计能够满足小王的愿望。

分析:设计的核心在于使 `copy` 函数能够灵活应对输入和输出设备的变化。根据需求分析,变化仅限于输入和输出,因此,可以将输入和输出独立抽象,并作为参数传递给 `copy` 函数。这样,`copy` 函数的实现就不受具体输入输出方式的影响,从而保持稳定性和灵活性。具体定义和实现如下:

```

1  //输入设备
2  class InputDevice {
3  public:
4      virtual int read( ) = 0;          // 纯虚函数,定义读取字符的方法
5      virtual ~InputDevice( ) = default; // 虚析构函数
6  };
7  class Keyboard : public InputDevice {
8  public:
9      int read( ) override {
10         // 模拟从键盘读取字符
11         int c = getchar( );
12         return c;
13     }
14 };
15 class TapeReader : public InputDevice {
16 public:
17     int read( ) override {
18         // 模拟从纸带读入机读取字符
19         int c = /* 从纸带读入机读取字符的逻辑 */;
20         return c;
21     }
22 };
23 //输出设备
24 class OutputDevice {
25 public:
26     virtual void write(int c) = 0; // 纯虚函数,定义写入字符的方法
27     virtual ~OutputDevice( ) = default; // 虚析构函数
28 };
29 class Printer : public OutputDevice {
30 public:
31     void write(int c) override {
32         // 模拟将字符输出到打印机
33         putchar(c);
34     }
35 };

```

```

36 class OtherOutputDevice : public OutputDevice {
37 public:
38     void write(int c) override {
39         // 模拟将字符输出到其他设备
40         /* 其他设备的输出逻辑 */
41     }
42 };
43 //重新设计实现的 Copy
44 void copy(InputDevice & input, OutputDevice & output) {
45     int c;
46     while ((c = input.read( )) != EOF) { // EOF 为宏定义的终结字符
47         output.write(c);
48     }
49 }
50 int main( ) {
51     Keyboard keyboard;
52     Printer printer;
53     // 从键盘读取并输出到打印机
54     Copy(keyboard, printer);
55
56     // 如果需要从纸带读入机读取并输出到其他设备
57     TapeReader tapeReader;
58     OtherOutputDevice otherOutput;
59     Copy(tapeReader, otherOutput);
60 }

```

19.4.7 图形输出

小王根据应用需求，定义并实现了一个适用于特殊格式的图形类(MyPic)，主要功能是根据给定的输出选项将图形输出到指定设备。以下是 MyPic 类的部分代码：

```

1 class MyPic {
2 public:
3     //根据指定的页边留白和对齐方式(居中/左对齐)等输出选项，输出到打印机
4     bool sendTo(Printer& printer, int marginLeft, int marginTop,
5         int marginRight, int marginBottom, int alignKind)
6     { /* 略 */ }
7     //根据指定的页边留白和对齐方式(居中/左对齐)等输出选项，输出到打印机
8     //其中一个 Point 对象表示一个具有(x,y)坐标的点
9     bool sendTo(Printer& printer, Point marginLT, Point marginRB, int alignKind)
10    { return SendTo(printer,marginLT.x,marginLT.y,marginRB.x,marginRB.y,alignKind); }
11    //根据指定的起始偏移、透明度和位操作掩码等输出选项，输出到位图
12    bool sendTo(Bitmap& bmp, int xOffset, int yOffset, int transparent, int mask)
13    { /* 略 */ }

```

```

14    // 其它略
15 };

```

对于 MyPic 类，小王发现重载的 sendTo 函数和冗长的参数列表影响了使用的便捷性。为了解决这一问题，小王计划通过抽象多个参数，消除函数重载，使类中仅保留一个 sendTo 函数，并且该函数只接受一个参数。请重新设计 MyPic 类，并考虑以下实际情况：

- (1) Printer、Bitmap 等类来自第三方类库，无法获取完整的源代码，因此不能直接从这些类派生新的子类；
- (2) 未来可能会增加新的输出选项，例如打印方向(横向/纵向)等；
- (3) 还将支持其他输出设备，如文件、视窗、晒图仪等。

分析：针对上述设计要求，可以通过将多个参数抽象为一个类来减少函数参数的数量，从而简化函数的使用。在本例中，可以考虑将参数列表抽象到一个公共基类 Parameters 中，类中的数据成员会根据具体的设计思路有所不同。最直接的方法是：将 Parameters 定义为接口类，而将不同形式的参数列表定义为不同的子类，每个子类中包含相应所需的数据成员。此时的主要类定义可如下：

```

1  class Parameters {
2  public:
3      virtual ~Parameters( ) = default;
4      virtual bool process( MyPic & pic ) = 0;
5  };
6  class PrinterWithOptions : public Parameters {
7  public:
8      PrinterWithOptions( Printer & prt, int left, int top, int right, int bottom
9          , int align ) : mpPrinter( &prt ), mLeft( left ), mTop( top )
10         , mRight( right ), mBottom( bottom ), mAlign( align ) { }
11      PrinterWithOptions( Printer & prt, Point lt, Point rb, int align )
12          :PrinterWithOptions( prt, lt.x, lt.y, rb.x, rb.y, align ) { }
13      bool process( MyPic & pic ) override { /*略*/ }
14  private:
15      Printer * mpPrinter;
16      int mLeft;
17      int mTop;
18      int mRight;
19      int mBottom;
20      int mAlign;
21  };
22  class MyPic {
23  public:

```

```

24     bool sendTo( Parameters & params ) {
25         return params.process( *this );
26     }
27 };
28 int main( ) {
29     MyPic pic;
30     Printer prt;
31     PrinterWithOptions prtOpts( prt, 1, 2, 3, 4, 0 );
32     pic.sendTo( prtOpts );
33 }

```

当需要扩展不同的输出设备或输出选项时,可以通过从 `Parameters` 类派生子类来实现。然而,这种简单的设计方式在面对输出设备与选项交叉组合的实际情况时,会导致派生子类的数量迅速增加,并且容易产生重复或相似的代码块。

因此,可以考虑对 `Parameters` 类进行改进。鉴于参数的变化主要集中在两个方面:输出设备和输出选项,我们可以将这两者分别抽象为独立的接口,并作为数据成员纳入 `Parameters` 类中。这样,输出设备和选项可以各自独立变化,而不会影响 `Parameters` 类的整体结构。改进后的类设计可能如下所示:

```

1  //输出设备接口类
2  class Device {
3  public:
4      virtual ~Device( ) = default;
5      virtual int deviceFunc( ) = 0;
6  };
7  //输出选项接口类
8  class Options {
9  public:
10     virtual ~Options( ) = default;
11     virtual int optionsFunc( ) = 0;
12 };
13 //抽象的参数列表
14 class Parameters {
15 public:
16     Parameters( Device & dev, Options & opts )
17         : mpDevice( &dev ), mpOptions( &opts ) {}
18     bool process( MyPic & pic ) {
19         /* 略 */
20     }
21 private:
22     Device * mpDevice;

```

```

23     Options * mpOptions;
24 };
25 class MyPic {
26 public:
27     bool sendTo( Parameters & params ) {
28         return params.process( *this );
29     }
30 };

```

接下来, 可以从 Device 类派生出不同的输出设备, 从 Options 类派生出不同的输出选项。例如:

```

1  //打印机输出设备
2  class PrinterDevice : public Device {
3  public:
4      PrinterDevice( Printer & prt ) : mpPrinter( &prt ) {}
5      int deviceFunc( ) override { /*略*/ }
6  private:
7      Printer * mpPrinter;
8  };
9  //位图输出设备
10 class BitmapDevice : public Device {
11 public:
12     BitmapDevice( Bitmap & bmp ) : mpBitmap( &bmp ) {}
13     int deviceFunc( ) override { /*略*/ }
14 private:
15     Bitmap * mpBitmap;
16 };
17 //页边距及对齐选项
18 class MarginOptions : public Options {
19 public:
20     MarginOptions( int left, int top, int right, int bottom, int align )
21         :mLeft( left ), mTop( top ), mRight( right ), mBottom( bottom ), mAlign( align ) {}
22     MarginOptions( Point lt, Point rb, int align )
23         :MarginOptions( lt.x, lt.y, rb.x, rb.y, align ) {}
24     int optionsFunc( ) override { /*略*/ }
25 private:
26     int mLeft;
27     int mTop;
28     int mRight;
29     int mBottom;
30     int mAlign;
31 };

```

19.5 虚机制的不足

虚机制适合适应单个维度的变化，但不适合多个维度的同时变化，这是虚机制的不足之处。为更好地说明这点，考虑如下的例子：

```

1  class Foo {
2  public:
3      void f( );
4      void g( );
5      void h( );
6  };

```

如果 $f()$ 有两种不同的实现， $g()$ 有三种不同的实现， $h()$ 有四种不同实现，如果只使用虚机制适应变化需求，那么可能需要从类 A 派生出 $2 \times 3 \times 4 = 24$ 个子类，特别是，当再增加一种 $f()$ 的实现时，需要再派生 $3 \times 4 = 12$ 个子类。这种适应变化的方式是不可忍受的！

实际上，类 Foo 中存在 $f()$ 、 $g()$ 和 $h()$ 在 3 个不同维度上的实现变化，而虚机制只能在一个维度上适应变化，所以本例中只使用虚机制是不合适的。为解决此问题，可以先将各维度的变化独立地抽象出来，再在单个维度的变化上使用虚机制和子类型化。此时新设计的 UML 类图可如图 19-2 所示。

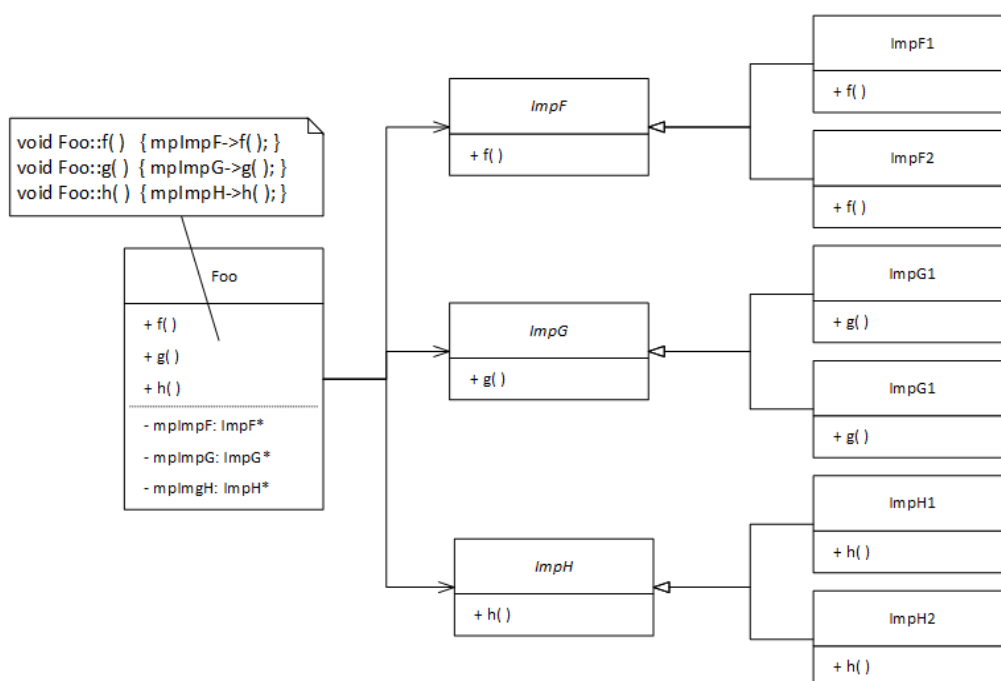


图 19-2 适应多维度变化示例的 UML 图

具体实现代码可能如下：

```
1 //将 f( )的实现变化单独抽象出来
2 class ImpF {
3     public:
4         virtual ~ImpF( ) { }
5         virtual void f( ) = 0;
6 };
7 class ImpF1 :public ImpF {
8     public:
9         virtual void f( ) override { /* f( )的第一种实现 */ }
10 };
11 class ImpF2 :public ImpF {
12     public:
13         virtual void f( ) override { /* f( )的第二种实现 */ }
14 };
15 //将 g( )的实现变化单独抽象出来
16 class ImpG {
17     public:
18         virtual ~ImpG( ) { }
19         virtual void g( ) = 0;
20 };
21 class ImpG1 :public ImpG {
22     public:
23         virtual void g( ) override { /* g( )的第一种实现 */ }
24 };
25 class ImpG2 :public ImpG {
26     public:
27         virtual void g( ) override { /* g( )的第二种实现 */ }
28 };
29 class ImpG3 :public ImpG {
30     public:
31         virtual void g( ) override { /* g( )的第三种实现 */ }
32 };
33 //将 h( )的实现变化单独抽象出来
34 class ImpH {
35     public:
36         virtual ~ImpH( ) { }
37         virtual void h( ) = 0;
38 };
39 class ImpH1 :public ImpH {
40     public:
41         virtual void h( ) override { /* h( )的第一种实现 */ }
42 };
43 class ImpH2 :public ImpH {
44     public:
```

```

45     virtual void h( ) override { /* h( )的第二种实现 */ }
46 };
47 class ImpH3 :public ImpH {
48 public:
49     virtual void h( ) override { /* h( )的第三种实现 */ }
50 };
51 class ImpH4 :public ImpH {
52 public:
53     virtual void h( ) override { /* h( )的第四种实现 */ }
54 };
55 //使用水平关系组织各维度的变化
56 class Foo {
57 public:
58     Foo( ImpF & impF, ImpG & impG, ImpH & impH )
59         :mpImpF( &impF ), mpImpG( &impG ), mpImpH( &impH ) { }
60     void f( ) { mpImpF->f( ); }
61     void g( ) { mpImpG->g( ); }
62     void h( ) { mpImpH->h( ); }
63 private:
64     ImpF * mpImpF;
65     ImpG * mpImpG;
66     ImpH * mpImpH;
67 };

```

总之，虚机制只适合适应单个维度的需求变化。当存在多个维度的变化时，可以使用依赖、普通关联、组合、聚合等关系先将各维度的变化分离出来，再针对单个维度使用子类型化和虚机制进行设计，以适应变化的需要。